Concepts of Programming Languages

Tail Calls and Continuations

Prof. Dr. Mira Mezini

Software Technology Group
TU Darmstadt | FB Informatik

Outline

- Recursion, Iteration, and Tail Call Optimization
- Short Journey in Continuation Passing Style (CPS)
- Continuations as First-Class Values in the Language
- Implementing Continuations
- Use Cases for Continuations

Recursive Factorial Function

```
def factorial1(n: Int): BigInt = {
  if (n == 1) 1
  else n * factorial1(n - 1)
}

println(factorial1(10000))
```

What do we get when we run this?

Recursive Factorial Function

```scala
def factorial1(n: Int): BigInt = {
  if (n == 1) 1
  else n * factorial1(n - 1)
```

```
java.lang.StackOverflowError
    at scala.math.BigInt$.apply(BigInt.scala:50)
    at scala.math.BigInt$.int2bigInt(BigInt.scala:118)
    at Playground$.factorial1(main.scala:5)
    at Playground$.factorial1(main.scala:5)
    at Playground$.factorial1(main.scala:5)
    at Playground$.factorial1(main.scala:5)
    at Playground$.factorial1(main.scala:5)
    at Playground$.factorial1(main.scala:5)
    at Playground$.factorial1(main.scala:5)
    at Playground$.factorial1(main.scala:5)
    at Playground$.factorial1(main.scala:5)
    at Playground$.factorial1(main.scala:5)
    at Playground$.factorial1(main.scala:5)
    at Playground$.factorial1(main.scala:5)
```

Stack Overflow!

Why?

Recursive Factorial Function

```
def factorial1(n: Int): Int = {
    if (n == 1) 1
    else n * factorial1(n - 1)
}

factorial1(6)
```

```
6 * factorial1(5)
6 * 5 * factorial1(4)
6 * 5 * 4 * factorial1(3)
6 * 5 * 4 * 3 * factorial1(2)
6 * 5 * 4 * 3 * 2 * factorial1(1)
6 * 5 * 4 * 3 * 2 * 1
6 * 5 * 4 * 3 * 2
6 * 5 * 4 * 6
6 * 5 * 24
6 * 120
720
```

Stack frames of function callers need to be stored and restored because there is still something to do after returning from the function call.

Recursive Factorial Function

```
def factIter(n: Int, result: BigInt): BigInt = {
  if (n <= 1) result
  else factIter(n - 1, result * n)
}
def factorial2(n: Int): BigInt = factIter(n, 1)

println(factorial2(10000))
```

What will we get when we run this program?

A huge number, no stack overflow!

Why?

A More Iterative Factorial Function

```
def factIter(n: Int, result: Int): Int = {
    if (n <= 1) result
    else factIter(n - 1, result * n)
  }

def factorial2(n: Int): Int = factIter(n, 1)


factorial2(6)
```

```
factIter(6, 1)
factIter(5, 6)
factIter(4, 30)
factIter(3, 120)
factIter(2, 360)
factIter(1, 720)
```

- Nothing interesting happens after function returns
- **No need to store and restore stack frame** of callers

Comparison

```
def factorial1(n: Int): Int = {
    if (n == 1) 1
    else n * factorial1(n - 1)
  }
```

Needs to return to caller; caller needs the result to complete

```
def factIter(n: Int, result: Int): Int = {
    if (n <= 1) result
    else factIter(n - 1, result * n)
  }

def factorial2(n: Int): Int = factIter(n, 1)
```

Captures the **rest of computation** and everything needed to complete it. No need to return.

Tail Call Optimization (TCO)

A call from a function `f` to a function `g` is a **tail call** if:
`f` simply returns the result of calling `g`
(in the control flow graph of `f`, the call of `g` is a leaf).

A special case is: **tail recursion** = a tail call from a function to itself.

Tail-call optimization (TCO):

- `g` sends its result directly to whoever is expecting `f`'s result
  ➔ **calling `g` does not require any stack manipulation**.
- Compilers apply TCO to replace calls in a tail position with jump statements ➔ saves stack space

Consequences of TCO

Function calls become inexpensive, same performance as inlined code

Recursive functions run on constant amount of stack space

Can use functions and recursion without stack-space or runtime penalty

Built-in looping constructs not absolutely necessary anymore (sugar)

Custom (User-Defined) Loop Constructs

- Built-in looping constructs not necessary anymore.
  - Whatever was previously written using a built-in loop can be written using tail recursion.
  - The compiler will convert the calls into gotos achieving the same efficiency as the loop version.

- Having custom loop constructs that are as efficient as build-in loops is interesting for new data structures.

A User-Defined `for`-Loop

```
def myfor(init: Int, pred: Int => Boolean, inc: Int => Int,
          body: (Int, Int) => Int, result: Int) =
{
    def loop(currentInd: Int, currentRes: Int): Int = {
      if (pred(currentInd))
         loop(inc(currentInd), body(currentInd, currentRes))
      else currentRes
    }

    loop(init, result)
}
```

`loop` contains a recursive tail call

`myfor` contains tail call to `loop`

A User-Defined `for`-Loop

```
def myfor(init: Int, pred: Int => Boolean, inc: Int => Int,
          body: (Int, Int) => Int, result: Int) =
{

    def loop(currentInd: Int, currentRes: Int): Int = {
      if (pred(currentInd))
         loop(inc(currentInd), body(currentInd, currentRes))
      else currentRes
    }

    loop(init, result)

}
```

```
myfor(10, { _ > 0 }, { _ - 1 }, { _ + _ }, 0)
```

What's the result?

Support for TCO in Programming Languages

- TCO is traditionally supported in functional languages like Scheme and ML.
- But any other language can have TCO as well!
- Some compilers support only tail recursion optimization - an important special case, but not the only useful one.
- Java (and similar languages) do not implement general tail call optimization because they want to preserve the stack (for stack traces and debugging purposes)

Outline

- Recursion, Iteration and Tail Call Optimization
- Short Journey in Continuation Passing Style (CPS)
- Continuations as First-Class Values in the Language
- Implementing Continuations
- Use cases for continuations

Have the Cake and Eat it Too?

- Functions with tail calls are more efficient
- But can be more complex to write

- `factorial1` :
  - \+ Code structure reflects mathematical definition
  - – Result depends on caller; hence we need a stack

- `factorial2`:
  - \+ Result does not depend on caller
  - – More complex definition (accidental complexity)

Can we have both clarity of definitions and efficiency of executions?

Continuation Passing Style (CPS)

- “Traditional code” can be transformed automatically into a form where **every call is a tail call**!
- Such a transformation is said to bring a function into **Continuation Passing Style** (**CPS**).

CPS as another semantics of function calls …

CPS as a Function Call Semantics

function `f` calls function `g` …

Traditional semantics

- `f` passes actual arguments to `g`
- `f` waits for `g` to return (eventually with a result)
- `f` resumes its computation eventually using the result of `g`

CPS Semantics

- In addition to arguments, `f` passes to `g` a function of one parameter, to which `g` should pass its result; this is the **continuation**.
- The **continuation** captures the **rest-of-computation** at the point of calling `g`.
- `g` does not return, but calls the continuation.

Factorial in CPS

Let‘s try to derive the CPS version of `factorial` …

```
def factorial1(n: Int): Int = {
    if (n == 1) 1
    else n * factorial1(n - 1)
}
```

CPS Semantics

- In addition to arguments, `f` passes to `g` a function of one parameter, to which `g` should pass its result; this is the **continuation**.
- The **continuation** captures the **rest-of-computation** at the point of calling `g`.
- `g` does not return, but calls the continuation.

Factorial in CPS

Let‘s try to derive the CPS version of `factorial` …

```
def factorial1(n: Int): Int = {
    if (n == 1) 1
    else n * factorial1(n - 1)
}
```

```
def factCPS(n: Int, k: Int => Int) {
    ???
}
```

1. Top-level factorial is called from somewhere with some continuation `k` . . .

Factorial in CPS: Questions to Answer

1. What should `factCPS` do in the case `n = 1`?
2. What's the rest of computation at the point of the recursive call?

```
def factCPS(n: Int, k: Int => Int) {
    ???
}
```

Factorial in CPS: Questions to Answer

1. What should `factCPS` do in the case `n = 1`?
2. What's the rest of computation at the point of the recursive call?

Instead of returning 1, pass it to k

```
def factCPS(n: Int, k: Int => Int) {
    if (n == 1) k(1)
    else ???
}
```

Factorial in CPS: Questions to Answer

- What should `factCPS` do in the case `n = 1`?
- What's the rest of computation at the point of the recursive call? So, what is the else branch of `factCPS`?

```
def factCPS(n: Int, k: Int => Int) {
    if (n == 1) k(1)
    else factCPS(n - 1, ???)
}
```

Rest-of-computation:
(a) multiply result of the recursive call, say `restResult`, with `n`
(b) pass the result of this multiplication to `k`

```
restResult => k(n * restResult)
```

Factorial in CPS

If `n` is not `1`, `factCPS` simply calls itself, passing the new continuation as a parameter.

```
def factCPS(n: Int, k: Int => Int) {
    if (n == 1) k(1)
    else factCPS(n - 1, { restResult => k(n * restResult) })
}
```

Factorial in CPS: Example Execution

```
def factCPS(n: Int, k: Int => Int) {
    if (n == 1) k(1)
    else factCPS(n - 1, { restResult => k(n * restResult) })
}
```

```
factCPS(3, {(x: Int) => x} )

factCPS(2, { res => ((x => x) (3 * res)) })

factCPS(2, { res => 3 * res })

factCPS(1, { res => ((res => 3 * res) (2 * res)) })

factCPS(1, { res => (3 * 2 * res) })

((res => (3 * 2 * res)) 1)

3 * 2 * 1 = 6
```

Systematic Transformation to CPS

- Any program can automatically transformed into a behaviorally equivalent version in CPS.
- Such transformation is called CPS transformation.

- See PLAI for details.

Outline

- Recursion, Iteration and Tail Call Optimization
- Short Journey in Continuation Passing Style (CPS)
- Continuations as First-Class Values in the Language
- Implementing Continuations
- Use Cases for Continuations

CPS Has its Problems …

1. CPS transformation requires access to the entire program: But sometimes we need to transform functions that are in a library, e.g., `map`.

2. Inhibits built-in compiler optimizations by replacing the stack with explicit representation of continuations

3. Assumes that the language has TCO to avoid the creation of needless stack frames.

First-Class Continuations

- Some programming languages provide an operation that reifies and returns the **"rest of the computation"** at any program point.

- Examples: Scheme, Smalltalk, Ruby, Standard ML.

In contrast to CPS: Program does not need to be transformed a priori. Reifications of rest-of-computation can be done on-the-fly.

First-Class Continuations

- In functional languages
  - “rest of the computation” is represented as a function of one argument called a **continuation.**
  - The computation can be resumed by applying the reified continuation to a value.

- In Smalltalk (OO language),
  - The “rest of computation” is represented as an object of type **`Context`**.
  - The computation is resumed by calling the method **`resume`** on that object.

Continuations in Scala

- Scala doesn't have native support for first-class continuations …
- Scala 2.11.12 (2016) had a continuation plugin, which can be still activated with Scastie… will use it to play around

For those of you interested:

- The following paper, presents how delimited continuations are implemented in Scala: "Implementing First-Class Polymorphic Delimited Continuations by a Type-Directed Selective CPS-Transform", by T. Rompf, I. Maier, M. Odersky
- https://github.com/scala/scala-continuations

Continuations in Scala

- The Scala continuation plugin supports two operations – **`shift`** and **`reset`**

- **`shift`** does two things:
  1. captures the rest of the execution (the continuation), at the point of its evaluation up to a region explicitly marked by the programmer via **`reset`** block
  2. passes the captured continuation as an argument to a higher-order function provided by the programmer.

```
reset {
    ...
    shift { k: (Int => Int) => ... }
    ...
}
```

Continuations in Scala

- The Scala continuation plugin supports two operations – **`shift`** and **`reset`**

- **`shift`** does two things:
  1. captures the rest of the execution (the continuation), at the point of its evaluation up to a region explicitly marked by the programmer via **`reset`** block
  2. passes the captured continuation as an argument to a higher-order function provided by the programmer.

```
reset {
   shift { k: (Int => Int) => k(3) } + 5
}
```

What happens here? What is the result of this program?

Examples of Programming With Continuations

1. The captured closure is a function `f` of one parameter that, when called with an argument `x`, adds 5 to `x`, `f(x) = x + 5`
2. The closure `f` is passed as argument to the function defined within the `shift` block `g(k) = k(3)`

So, the result is 8

```
reset {
   shift { k: (Int => Int) => k(3) } + 5
}
```

What happens here? What is the result of this program?

Examples of Programming With Continuations

```
def foo() = shift{k: (Int => Int) => k(7) + 1}
println(reset{2 * foo()})
```

`k = f(x) = 2 * x`

`k` is passed to `g(k) = k(7) + 1`.

`g(k) = k(7) + 1 = (2 * 7) + 1 = 15`

```
println(reset {shift { k: (Int=>Int) => k(k(k(7))) } + 1 } * 2)
```

`K = f(x) = x + 1`

`K` is passed to `g(K) = K(K(K(7)))`

`g(K) = K(K(K(7))) = (((7 + 1) + 1) + 1) = 10`

`println(10 *2)` → `20`

Outline

- Recursion, Iteration and Tail Call Optimization
- Short Journey in Continuation Passing Style (CPS)
- Continuations as First-Class Values in the Language
- Implementing Continuations
- Use Cases for Continuations

Implementing Continuations

Any idea as how we can implement a language with support for first-class continuations?

Implementing Continuations

- The interpreter is itself a program; hence, it can also be transformed to CPS.

- In a CPS interpreter the current continuation is available at each stage of the interpretation ➔ it can be provided to the programmer, if required.

- **General approach to implementing continuations**
  1. Make them explicit in the interpreter.
  2. Provide access to them in an extended language.

Let‘s turn the CFWAE interpreter into CPS …

KCFWAE – The CFAE Interpreter in CPS

```
def interp(expr:KCFLAE, env:Env, k: (Value => Nothing) ): Nothing = {
expr match {
  ...
  ...
  ...
  }
}
```

Continuation expects the result of interpreting each expression:

1. If the interpreter has a value handy, supply it to the continuation
2. Otherwise, the interpreter calls itself recursively with a (possibly augmented) continuation

The Case of Number, Identifiers, Closures

```
def interp(expr:KCFLAE, env:Env, k: (Value => Nothing) ): Nothing = {
expr match {
  ...
  case Num(n)            => ???

  case Id(name)          => ???

  case Fun(arg, body)    => ???

  ...
  }
}
```

What should we do here?

The Case of Number, Identifiers, Closures

```
def interp(expr:KCFLAE, env:Env, k: (Value => Nothing) ): Nothing = {
expr match {
  ...
  case Num(n)          => k (NumV(n))

  case Id(name)        => k (env(name))

  case Fun(arg, body)  => k (Closure(arg, body, env))
  ...

  }
}
```

Values are simply fed to continuation

CPS Interpretation of Non-Value Expressions

In all other cases:

Further steps of interpretation are performed by recursive calls with a new augmented continuation.

All pending computation must be moved to the continuation!

Rule of thumb:

A call to the `interp` function never appears as a sub-expression in the definition of the `interp` function.

It's always the first thing we do…

What needs to be done for CPS interpretation of arithmetic operations?

Interpretation of Arithmetic OPs

```
def interp(expr:KCFLAE, env:Env, k: (Value => Nothing) ): Nothing = {
expr match {


  ...
  case Add(lhs,rhs) =>
      interp(lhs, env,




      )
...
```

Augmented continuation for interpreting the right-hand operand

Augmented continuation for interpreting the left-hand operand

Interpretation of Arithmetic OPs

```
def interp(expr:KCFLAE, env:Env, k: (Value => Nothing) ): Nothing = {
expr match {


  ...
  case Add(lhs,rhs) =>
      interp(lhs, env, lv => {
          interp(rhs, env, rv =>
                (lv, rv) match {
                  case (NumV(n1), NumV(n2)) => k(NumV(n1 + n2))
                  }
                )
          }
      )
...
```

Augmented continuation for interpreting the right-hand operand

Augmented continuation for interpreting the left-hand operand

CPS Interpretation of Conditionals

```
def interp(expr:KCFLAE, env:Env, k: (Value => Nothing) ): Nothing = {
expr match {
  ...
   case If0(c, t, e) =>
   {
      interp(c, env, cv =>
        cv match {
          case NumV(0) => interp(t, env, k )
          case _       => interp(e, env, k )
        })
    }
  }
}
```

Do we need to augment the continuation for interpreting **then** and **else** expressions?

No: The pending computation after evaluating any of the branches is the same as the one of the **If0** expression.

A quick reflection on tail calls …

Tail Calls Revisited

- Converting a function to CPS makes the stack explicit.
- We can see if a function invocation requires something “to be pushed to the stack”.
- If a function, `f`, invokes another one, `g`, with the **same continuation** that it received, `f` doesn’t push anything to the stack.
- **Such invocations were tail calls before the CPS transformation.**

Tail Calls Revisited

An example that was a tail call before conversion

```
interp( ... k ...
 ...
 case If0(c, t, e) =>
 {
   interp(c, env, (cv) =>
     cv match {
       case NumV(0) => interp(t, env, k )
       case _       => interp(e, env, k )
     })
 }
```

```
case If0(testExpr, thenExpr, elseExpr) => {
      val testV = interp(testExpr, funDefs, subRep)
      if (testV == 0)
            interp(thenExpr, funDefs, subRep)
          else
            interp(elseExpr, funDefs, subRep)
    }
```

Tail Calls Revisited

The calls in the implementation of addition have not been tail calls, since they augment the original continuation:

```
case Add(lhs,rhs) =>
      interp(lhs, env,
        lv => {
           interp(rhs, env,
             rv => (lv, rv) match {
               case (NumV(n1), NumV(n2)) => k(NumV(n1 + n2))
        })
      })
```

Back to the CPS transformation of the interpreter function …

What are the steps for CPS interpretation of application expressions?

CPS Interpretation of Application

```
def interp(expr: KCFLAE, env: Env, k: (Value => Nothing)): Nothing = {
  …
  case App(funExpr, argExpr) => {
        interp(funExpr, env, fv => {
          interp(argExpr, env, argV => fv match {
            case Closure(param, body, funEnv) => {
              interp(body, funEnv + (param -> argV), k)
            }
            case _ => {
              sys.error("Can only apply closures but got %s")
            }
          })
        })
      }
```

continuation for interpreting the application is the dynamic continuation for interpreting body of the closure

We have transformed the interpreter so that the current continuation is available at each interpretation step…

What do you think, should we do to add continuations to the language?

Making Continuations Available

1. We add a new value type for first-class continuations. Represented by Scala functions.

2. We add the `BindCC` construct, which binds the current continuation to a name. `BindCC` corresponds to `let/cc` in Scheme (non-delimited `shift` in Scala)

3. We need to define how to apply a continuation.

1. Add Continuation Values

1. We add a new value type for first-class continuations. Represented by Scala functions.

2. We add the `BindCC` construct, which binds the current continuation to a name. `BindCC` corresponds to `let/cc` in Scheme (non-delimited `shift` in Scala)

3. We need to define how to apply a continuation.

```
sealed abstract class Value
case class NumV(n: Int) extends Value
case class Closure(param: Symbol, body: KCFLAE, env: Env) extends Value
case class Continuation(c: (Value => A)) extends Value
```

Making Continuations Available

1. We add a new value type for first-class continuations. Represented by Scala functions.

2. We add the `BindCC` construct, which binds the current continuation to a name. `BindCC` corresponds to `let/cc` in Scheme (non-delimited `shift` in Scala)

3. We need to define how to apply a continuation.

2. Adding `BindCC`

```
<KCFAE> ::=
        ...
        | {BindCC <id> <KCFAE>}
        ...
```

What needs to be done in the interpreter?

```
def interp(expr: KCFLAE, env: Env, k: (Value => Nothing)): Nothing =
{
    ...

    case BindCC(name, body) => ???

    ...

}
```

Interpreting `BindCC`

Interpret **`body`** in the current environment at continuation reification time **`env`** extended with a binding for **`name`** to the current continuation value at the point of interpreting **`bindCC`**

What should we put as the value of the continuation?

```
case BindCC(name, body) =>
     interp(body, env + (name -> ??? ), k )
```

The continuation for interpreting the body is **`k`**

Interpreting `BindCC`

Interpret **`body`** in the current environment at continuation reification time **`env`** extended with a binding for **`name`** to the current continuation value at the point of interpreting **`bindCC`**

```
case BindCC(name, body) =>
        interp(body, env + (name -> Continuation(k)), k)
```

Making Continuations Available

1. We add a new value type for first-class continuations. Represented by Scala functions.

2. We add the `BindCC` construct, which binds the current continuation to a name. `BindCC` corresponds to `let/cc` in Scheme (non-delimited `shift` in Scala)

3. We need to define how to apply a continuation.

We will overload function application
to handle continuation application.

Overloading Function Application

```
def interp(expr: KCFLAE, env: Env, k: (Value => Nothing)): Nothing = {
  …
  case App(funExpr, argExpr) => {
        interp(funExpr, env, fv => {
          interp(argExpr, env, argV =>
            fv match {
              case Closure(param, body, funEnv) => {
                  interp(body, funEnv + (param -> argV), k)}
              case Continuation(c) => ???
              case _ => {
                sys.error("Can only apply closures but got %s")
              }
            })
          })
   }
...
```

Overloading Function Application

```
def interp(expr: KCFLAE, env: Env, k: (Value => Nothing)): Nothing = {
  …
  case App(funExpr, argExpr) => {
        interp(funExpr, env, fv => {
          interp(argExpr, env, argV =>
            fv match {
              case Closure(param, body, funEnv) => {
                  interp(body, funEnv + (param -> argV), k)}
              case Continuation(c) => c(argV)
              case _ => {
                sys.error("Can only apply closures but got %s")
              }
```

The whole point about continuations is to ignore the current continuation and use the stored one instead!

Unlike for closures, the interpretation of continuations does not take a continuation as an argument. Why?

…

Adding Continuations to the Language

Interpreter continuations capture what's left to be done in the interpreter, not in the program.

- but, the interpreter simulates the execution of the program
- so, the continuation of the interpreter simulates the corresponding continuation of the program

Initial Continuation

- What is the initial value of `k` to pass to the interpreter?
- Theoretically it is "the rest of Scala runtime"
- Practically, we can use the identity function, or a function that prints the received value and terminates.

Interim Summary

- Tail calls and TCO enable the use of functions and recursion without stack-space or run-time penalty.
- Programs that only contain tail calls may have a more complex structure than equivalent programs without tail calls.
- Any program can be automatically transformed to a CPS version that only contains tail calls.
- There are languages with built-in support for continuations, e.g., Scheme
- To implement: (a) Transform the interpreter to CPS; (b) Make continuation available at any point of the interpreter execution via a special binding construct.

Outline

- Recursion, Iteration and Tail Call Optimization
- Short Journey in Continuation Passing Style (CPS)
- Continuations as First-Class Values in the Language
- Implementing Continuations
- Use Cases for Continuations

Common Use Cases for Continuations

Advanced Exception Handling
Capture the continuation at the point of an exception. Resume the execution of the program later from that point or perform custom exception handling.

Undo Mechanisms
By capturing the state of the program at various points, it becomes possible to roll back to a previous state when needed.

Asynchronous Programming Continuations can be used to suspend / resume the execution of a function at specific points, enabling handling of asynchronous tasks without blocking the entire program.

Common Use Cases for Continuations

Dynamic Control Flow Manipulation
Continuations allow for non-local jumps in the execution flow, which can be useful in scenarios where traditional control flow constructs are not sufficient.

Stateful Web Programming
Continuations can be used to capture the state of a web application at a specific point to enable features like "back" buttons, where the user can navigate to a previous state of the application.